

SKRIPTA ZA UČENJE

Družbalica - full-stack katalog maskota

ASP.NET Core · Entity Framework Core · SQL Server · REST API · React · TypeScript

5. avgust 2026.

1. Šta smo napravili

Danas smo napravili prvu kompletnu funkcionalnu celinu aplikacije Družbalica: katalog maskota.

Tok podataka izgleda ovako:

TEXT

SQL Server baza

↓ Entity Framework Core

ASP.NET Core Web API

↓ HTTP + JSON

React frontend

↓

Katalog maskota i stranica detalja

Backend može da pročita, doda, izmeni i obriše maskotu. Frontend trenutno koristi operacije čitanja da prikaže katalog i detalje izabrane maskote.

2. Podela aplikacije na slojeve

Aplikacija ima nekoliko delova sa različitim odgovornostima:

- Baza podataka trajno čuva podatke.
- Entity Framework Core povezuje C# objekte i tabele u bazi.
- Web API prima HTTP zahteve, izvršava logiku i vraća odgovor.
- DTO klase definišu podatke koje API prima i vraća.
- React frontend poziva API i prikazuje podatke korisniku.

Važno je da svaki sloj radi svoj posao. React ne pristupa direktno bazi, već komunicira sa API-jem. API je centralno mesto za pristup podacima i poslovna pravila.

3. Model Mascot

Model predstavlja maskotu u aplikaciji i osnovu za tabelu `Mascots`.

Najvažnija polja su:

- `Id` — jedinstveni identifikator i primarni ključ;
- `Name` — naziv maskote;
- `Description` — opis;
- `ImageUrl` — adresa slike;
- `RentalPrice` — cena iznajmljivanja;
- `SalePrice` — opciona prodajna cena;
- `IsAvailableForRent` — da li je dostupna za najam;
- `IsAvailableForSale` — da li je dostupna za prodaju;
- `StockQuantity` — količina na stanju.

Tipovi podataka

- `int` koristimo za cele brojeve.
- `string` koristimo za tekst.
- `bool` ima vrednost `true` ili `false`.
- `decimal` koristimo za novac jer je precizniji i pogodniji od `float` ili `double`.
- `decimal?` znači da vrednost može biti broj ili `null`. To ima smisla za `SalePrice`, jer maskota ne mora biti na prodaju.

Data annotations

Atributi iz `System.ComponentModel.DataAnnotations` opisuju pravila validacije:

- `[Required]` — vrednost je obavezna;
- `[MaxLength(100)]` — tekst može imati najviše 100 znakova;
- `[Range(0, ...)]` — broj ne sme biti negativan.

Zbog atributa `[ApiController]`, ASP.NET Core automatski proverava ulazni model. Ako zahtev nije validan, API može automatski vratiti status `400 Bad Request` pre ulaska u metodu kontrolera.

4. ApplicationDbContext i Entity Framework Core

`ApplicationDbContext` predstavlja vezu između aplikacije i baze podataka. Nasleđuje EF Core klasu `DbContext`.

CSHARP

```
public DbSet<Mascot> Mascots => Set<Mascot>();
```

`DbSet<Mascot>` predstavlja skup maskota nad kojim možemo da izvršavamo upite i izmene. U praksi odgovara tabeli `Mascots`.

U `OnModelCreating` dodatno podešavamo šemu baze:

- maksimalne dužine tekstualnih kolona;
- preciznost cena `decimal(18,2)`.

`decimal(18,2)` znači ukupno najviše 18 cifara, od kojih su dve iza decimalnog zareza.

Registracija konteksta

U `Program.cs` kontekst se registruje u sistemu za dependency injection:

CSHARP

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"));
```

Time govorimo aplikaciji da koristi SQL Server i connection string pod nazivom `DefaultConnection`.

Dependency injection

Kontroler ne pravi ručno novi `ApplicationDbContext`. Prima ga kroz konstruktor:

CSHARP

```
public MascotsController(ApplicationDbContext context)
{
    _context = context;
}
```

ASP.NET Core automatski kreira i prosleđuje potrebnu instancu. To se zove dependency injection. Prednosti su manje povezivanja između klasa, lakše testiranje i centralna konfiguracija zavisnosti.

5. Migracije

Migracija je verzionisana promena strukture baze. Početna migracija `InitialCreate` pravi tabelu `Mascots` sa kolonama izvedenim iz modela i EF konfiguracije.

Migracija ima dve važne metode:

- `Up()` primenjuje promenu, ovde pravi tabelu;
- `Down()` poništava promenu, ovde briše tabelu.

Tipičan tok je:

POWERSHELL

```
dotnet ef migrations add NazivMigracije
dotnet ef database update
```

Prva komanda generiše opis promene, a druga je primenjuje na bazu. Migracije treba čuvati u Git-u jer predstavljaju istoriju šeme baze.

6. REST API i CRUD

CRUD označava četiri osnovne operacije nad podacima:

Operacija	HTTP metoda	Ruta	Rezultat
Create	POST	/api/mascots	Dodaje maskotu
Read all	GET	/api/mascots	Vraća katalog
Read one	GET	/api/mascots/{id}	Vraća jednu maskotu
Update	PUT	/api/mascots/{id}	Menja maskotu
Delete	DELETE	/api/mascots/{id}	Briše maskotu

`[Route("api/[controller]")]` koristi ime kontrolera bez reči `Controller`, pa `MascotsController` dobija osnovnu rutu `/api/mascots`.

HTTP statusni kodovi koje smo koristili

- **200 OK** — zahtev za čitanje je uspešan;
- **201 Created** — novi resurs je napravljen;
- **204 No Content** — izmena ili brisanje je uspešno, bez tela odgovora;
- **400 Bad Request** — ulaz nije validan;
- **404 Not Found** — tražena maskota ne postoji.

Statusni kod je deo ugovora API-ja. Klijent na osnovu njega zna da li je zahtev uspeo i šta se dogodilo.

7. DTO klase

DTO znači Data Transfer Object. To je klasa namenjena prenosu podataka kroz API, a ne direktnom predstavljanju reda u bazi.

Koristili smo četiri DTO-a:

- `CreateMascotRequest` — podaci dozvoljeni pri kreiranju;
- `UpdateMascotRequest` — podaci dozvoljeni pri izmeni;
- `MascotCatalogItemResponse` — sažeti podaci za karticu u katalogu;
- `MascotDetailsResponse` — prošireni podaci za stranicu detalja.

Zašto ne vraćamo svuda bazni model

DTO-i:

- sprečavaju klijenta da menja polja koja ne bi smeo;
- odvajaju API ugovor od strukture baze;
- omogućavaju da različiti ekrani dobiju samo potrebne podatke;
- smanjuju količinu podataka u odgovoru;
- čine nameru endpointa jasnijom.

Pretvaranje request DTO-a u entitet zove se mapiranje. U ovom projektu mapiranje je trenutno ručno, što je sasvim dovoljno i pregledno za mali broj polja.

8. Čitanje podataka pomoću LINQ-a

Primer upita za katalog:

CSHARP

```
var mascots = await _context.Mascots
    .AsNoTracking()
    .Select(mascot => new MascotCatalogItemResponse { /* polja */ })
    .ToListAsync();
```

Delovi upita:

- `_context.Mascots` bira tabelu;
- `AsNoTracking()` kaže EF-u da ne prati objekte jer ih samo čitamo;
- `Select(...)` projektuje samo potrebna polja u DTO;
- `ToListAsync()` asinhrono izvršava upit i vraća listu.

Za detalje jedne maskote koristimo:

- `Where(mascot => mascot.Id == id)` za filtriranje;

- `FirstOrDefaultAsync()` za prvi rezultat ili `null` ako ga nema.

LINQ izraz EF Core prevodi u SQL. Upit se ne izvršava odmah posle `_context.Mascots`, već tek kod završne operacije poput `ToListAsync()` ili `FirstOrDefaultAsync()`.

9. Kreiranje, izmena i brisanje

Kreiranje

Kod kreiranja:

- 1 API prima `CreateMascotRequest` iz JSON tela zahteva.
- 2 ASP.NET Core proverava validaciju.
- 3 DTO se mapira u novi `Mascot` entitet.
- 4 `_context.Mascots.Add(mascot)` označava entitet za dodavanje.
- 5 `SaveChangesAsync()` izvršava `INSERT` u bazi.
- 6 Baza dodeljuje `Id`.
- 7 API vraća `201 Created` i lokaciju novog resursa pomoću `CreatedAtAction`.

Izmena

`FindAsync(id)` pronalazi postojeći entitet. Ako ne postoji, vraćamo `404`. Ako postoji, menjamo njegova polja i pozivamo `SaveChangesAsync()`. EF Core prati entitet i generiše odgovarajući `UPDATE`.

PUT u ovom slučaju predstavlja potpunu zamenu izmenjivih podataka maskote. Za delimične izmene bi se mogao koristiti `PATCH`, ali on ovde nije potreban.

Brisanje

Posle pronalaženja entiteta pozivamo `Remove(mascot)`, a zatim `SaveChangesAsync()`. Ako entitet ne postoji, vraćamo `404`; uspešno brisanje vraća `204`.

10. Asinhrono programiranje

Metode kontrolera su označene sa `async`, a operacije nad bazom čekamo pomoću `await`:

CSHARP

```
public async Task<IActionResult> GetMascots()
```

Dok aplikacija čeka bazu, serverska nit ne mora beskorisno da bude blokirana. Server zato može efikasnije da obrađuje druge zahteve. `Task<IActionResult>` predstavlja operaciju koja će kasnije proizvesti HTTP rezultat.

Asinhroni pozivi ne znače automatski paralelno izvršavanje; oni prvenstveno omogućavaju efikasno čekanje I/O operacija kao što su baza i mreža.

11. Program.cs i HTTP pipeline

U `Program.cs` registrujemo servise, gradimo aplikaciju i određujemo redosled middleware komponenti.

Važne stavke su:

- `AddControllers()` — uključuje kontrolere;
- `AddOpenApi()` — generiše OpenAPI opis u development okruženju;
- `AddDbContext(...)` — registruje bazu;
- `AddCors(...)` — definiše ko sme da poziva API iz browsera;

- `UseHttpsRedirection()` — preusmerava HTTP na HTTPS;
- `UseCors("Frontend")` — uključuje izabranu CORS politiku;
- `UseAuthorization()` — uključuje autorizacioni middleware;
- `MapControllers()` — povezuje rute sa kontrolerima.

Redosled middleware-a je važan jer zahtev prolazi kroz pipeline redom kojim su komponente registrovane.

12. CORS

Frontend i backend rade na različitim originima, na primer:

TEXT

```
Frontend: http://localhost:3000
Backend: http://localhost:5004
```

Origin čine protokol, domen i port. Pošto je port različit, browser ih smatra različitim originima i primenjuje CORS zaštitu.

Backend zato eksplicitno dozvoljava frontend origin, zaglavlja i HTTP metode. CORS nije mehanizam autentifikacije; to je browser pravilo koje kontroliše međusobne zahteve različitih origin-a.

13. React i komponenta kataloga

Početna stranica je React funkcionalna komponenta. Direktiva:

TSX

```
"use client";
```

znači da komponenta koristi browser mogućnosti i React hook-ove na klijentu.

TypeScript tip

`MascotCatalogItem` opisuje oblik JSON objekta koji frontend očekuje od API-ja. TypeScript proverava upotrebu podataka tokom razvoja, ali ne proverava stvarni odgovor servera u runtime-u.

useState

Komponenta čuva tri vrste stanja:

- `mascots` — učitani podaci;
- `isLoading` — da li učitavanje traje;
- `error` — poruka o grešci ili `null`.

Promena state-a izaziva ponovno renderovanje komponente.

useEffect

`useEffect(..., [])` se pokreće nakon prvog renderovanja komponente. Unutar njega pozivamo API.

Prazan niz zavisnosti znači da efekat nije vezan za promenljivu koja bi zahtevala novo učitavanje. U razvojnom režimu React može pokrenuti efekat više puta radi otkrivanja problema, ali produkciono ponašanje nije isto.

fetch

`fetch` šalje HTTP zahtev:

TSX

```
const response = await fetch(`${apiUrl}/api/mascots`);
```

Važno: `fetch` ne baca grešku automatski za statuse kao što su `404` ili `500`. Zato proveravamo `response.ok`. Zatim `response.json()` pretvara JSON telo u JavaScript podatke.

`try/catch/finally` deli tok na:

- `try` — pokušaj učitavanja;
- `catch` — mrežna ili ručno bačena greška;
- `finally` — završetak loading stanja bez obzira na ishod.

14. Renderovanje liste i uslovni prikaz

Listu prikazujemo metodom `map`:

TSX

```
mascots.map((mascot) => (  
  <article key={mascot.id}>...</article>  
))
```

`key` pomaže React-u da stabilno razlikuje elemente liste. Treba da bude jedinstven i stabilan, zato koristimo ID iz baze.

Komponenta razlikuje četiri UI stanja:

- 1 učitavanje;
- 2 greška;
- 3 uspešan odgovor bez rezultata;
- 4 uspešan odgovor sa maskotama.

To je važan obrazac: ekran nije samo „podaci postoje“, već mora pravilno prikazati svaki mogući ishod asinhronne operacije.

`toLocaleString("sr-RS")` formatira broj prema srpskom lokalitetu, a ne menja samu vrednost cene.

15. Rutiranje i stranica detalja

Putanja:

TEXT

```
app/mascots/[id]/page.tsx
```

predstavlja dinamičku rutu. Deo `[id]` se menja u zavisnosti od izabrane maskote, na primer `/mascots/3`.

`Link` omogućava navigaciju unutar aplikacije. Za razliku od običnog punog učitavanja stranice, framework može efikasnije da obavi klijentsku navigaciju.

`useParams<{ id: string }>()` čita parametar iz URL-a. Parametar je string jer dolazi iz putanje.

Efekat na stranici detalja zavisi od `params.id`:

TSX

```
useEffect(() => {  
  // učitaj maskotu  
}, [params.id]);
```

Ako se ID promeni dok je ista komponenta aktivna, podaci se ponovo učitavaju.

Frontend posebno obrađuje 404 i prikazuje korisniku da maskota ne postoji. Ostale neuspešne odgovore prikazuje kao opštu grešku.

16. Konfiguracija API adrese

Frontend čita adresu backend-a iz environment promenljive:

TSX

```
const apiUrl = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:5004";
```

Promenljive sa prefiksom `NEXT_PUBLIC_` smeju da budu ugrađene u klijentski kod. Zato u njima ne sme biti tajni, lozinki ni privatnih ključeva.

Operator `??` koristi rezervnu vrednost samo kada je leva strana `null` ili `undefined`.

Ovakva konfiguracija omogućava da lokalno koristimo localhost, a na hostingu adresu objavljenog API-ja bez promene izvornog koda.

17. Veza između backenda i frontenda

Primer toka pri otvaranju kataloga:

- 1 Browser renderuje početnu React komponentu.
- 2 `useEffect` poziva `GET /api/mascots`.
- 3 Zahtev prolazi kroz ASP.NET Core pipeline.
- 4 Ruta bira `GetMascots` metodu.
- 5 EF Core prevodi LINQ u SQL i čita bazu.
- 6 Entiteti se projektuju u `MascotCatalogItemResponse` DTO-e.
- 7 ASP.NET Core serijalizuje DTO listu u JSON.
- 8 React pretvara JSON u niz objekata i poziva `setMascots`.
- 9 Komponenta se ponovo renderuje i prikazuje kartice.

Za detalje je tok isti, osim što URL sadrži ID, API filtrira po njemu i vraća jedan objekat ili 404.

18. JSON i imenovanje polja

C# svojstva koriste PascalCase, na primer `RentalPrice`, dok JSON koji frontend dobija koristi camelCase, na primer `rentalPrice`. ASP.NET Core podrazumevano radi ovu konverziju pri JSON serijalizaciji.

Primer odgovora:

JSON

```
{
  "id": 1,
  "name": "Meda",
  "description": "Maskota za dečje proslave",
  "imageUrl": "",
  "rentalPrice": 5000,
  "salePrice": null,
  "isAvailableForRent": true,
  "isAvailableForSale": false
}
```


API ugovor mora da se slaže sa TypeScript tipom na frontendu. Ako se DTO promeni, treba proveriti i frontend koji ga koristi.

19. Organizacija po funkcionalnosti

Fajlovi za maskote grupisani su u `Features/Mascots`. To se zove organizacija po funkcionalnosti: model, DTO-i i kontroler jedne celine nalaze se zajedno.

Za trenutnu veličinu projekta ova struktura je jednostavna i pregledna. Nema potrebe unapred uvoditi dodatne slojeve, repozitorijume ili komplikovane obrasce bez stvarne potrebe.

Uklonjen je i početni `WeatherForecast` primer zato što više nije deo domena aplikacije. Šablonski kod treba ukloniti kada prestane da bude koristan, kako ne bi stvarao zabunu.

20. Najvažnije razlike koje treba znati

Entitet i DTO

- Entitet predstavlja podatak kojim EF Core upravlja u bazi.
- DTO predstavlja podatak koji prelazi granicu API-ja.

IEnumerable rezultat i izvršavanje upita

Sastavljanje LINQ izraza nije isto što i izvršavanje SQL upita. Završne asinhronne metode, poput `ToListAsync`, zaista šalju upit bazi.

Add i SaveChangesAsync

`Add` samo označava objekat za dodavanje u EF kontekstu. Tek `SaveChangesAsync` trajno upisuje promenu u bazu.

null i prazna vrednost

- `null` znači da vrednost ne postoji.
- `""` je string koji postoji, ali nema znakove.
- `0` je bročana vrednost, nije isto što i `null`.

Validacija i poslovna pravila

Validacija proverava oblik i osnovne granice podataka, na primer obavezno ime i nenegativnu cenu. Poslovno pravilo proverava smisao za konkretan domen, na primer da se dve potvrđene rezervacije iste maskote ne preklapaju. Takvo pravilo još nije deo današnjeg kataloga.

21. Kako ručno proveriti funkcionalnost

Backend i frontend treba pokrenuti odvojeno.

Tipične provere API-ja:

- 1 `GET /api/mascots` vraća listu.
- 2 `POST /api/mascots` sa validnim JSON-om vraća `201`.
- 3 `POST` sa praznim imenom ili negativnom cenom vraća `400`.
- 4 `GET /api/mascots/{id}` vraća dodat objekat.
- 5 `GET` za nepostojeći ID vraća `404`.
- 6 `PUT` menja podatke i vraća `204`.

7 `DELETE` briše podatak i vraća `204`.

Tipične provere frontenda:

- 1 katalog prikazuje podatke iz baze;
- 2 prikazuje se loading stanje;
- 3 gašenje API-ja prikazuje poruku o grešci;
- 4 prazan katalog prikazuje prazno stanje;
- 5 klik na karticu otvara pravilan ID;
- 6 nepostojeći ID prikazuje poruku „nije pronađeno“;
- 7 dugme za zahtev je onemogućeno kada maskota nije dostupna.

22. Pitanja za proveru znanja

- 1 Zašto React ne treba direktno da pristupa SQL Server bazi?
- 2 Šta predstavlja `DbContext`, a šta `DbSet<Mascot>`?
- 3 Koja je razlika između modela `Mascot` i `CreateMascotRequest`?
- 4 Zašto je `decimal` dobar izbor za cenu?
- 5 Šta znači znak `?` u `decimal`??
- 6 Čemu služe migracije i koja je razlika između `Up` i `Down`?
- 7 Koje HTTP metode odgovaraju CRUD operacijama?
- 8 Kada vraćamo `201`, kada `204`, a kada `404`?
- 9 Zašto koristimo `AsNoTracking` pri čitanju?
- 10 Kada se LINQ upit zaista izvršava nad bazom?
- 11 Šta rade `async` i `await`?
- 12 Zašto je potreban CORS kada frontend i backend imaju različite portove?
- 13 Koja je razlika između `useState` i `useEffect`?
- 14 Zašto proveravamo `response.ok` posle `fetch` poziva?
- 15 Čemu služi `key` u React listi?
- 16 Kako `[id]` u imenu foldera postaje dinamička ruta?
- 17 Zašto `NEXT_PUBLIC_API_URL` ne sme sadržati tajnu?
- 18 Opiši ceo tok od otvaranja kataloga do prikaza podataka iz baze.

23. Kratak odgovor za razgovor za posao

„Napravila sam prvu vertikalnu funkcionalnost full-stack aplikacije za upravljanje maskotama. Na backendu sam koristila ASP.NET Core Web API, Entity Framework Core i SQL Server. Definisala sam `Mascot` entitet, konfiguraciju baze i početnu migraciju, a zatim CRUD endpoint-e sa validacijom i posebnim request i response DTO klasama. Upiti za čitanje koriste asinhroni LINQ, projekciju i `AsNoTracking`. Na React frontendu sam napravila katalog i dinamičku stranicu detalja, sa TypeScript tipovima, `fetch` pozivima, loading, empty i error stanjima. Frontend i API su povezani preko JSON-a, a pristup iz browsera je omogućen kontrolisanom CORS politikom.“

24. Suština današnje lekcije

Najvažnije nije samo to što katalog radi, već razumevanje granica sistema:

TEXT

React prikazuje i šalje zahteve
API kontroliše pristup i oblik podataka
EF Core prevodi rad sa objektima u SQL
SQL Server trajno čuva stanje

Kada razumeš kako zahtev prolazi kroz sva četiri dela i vraća se kao odgovor, razumeš osnovu današnje full-stack celine.